



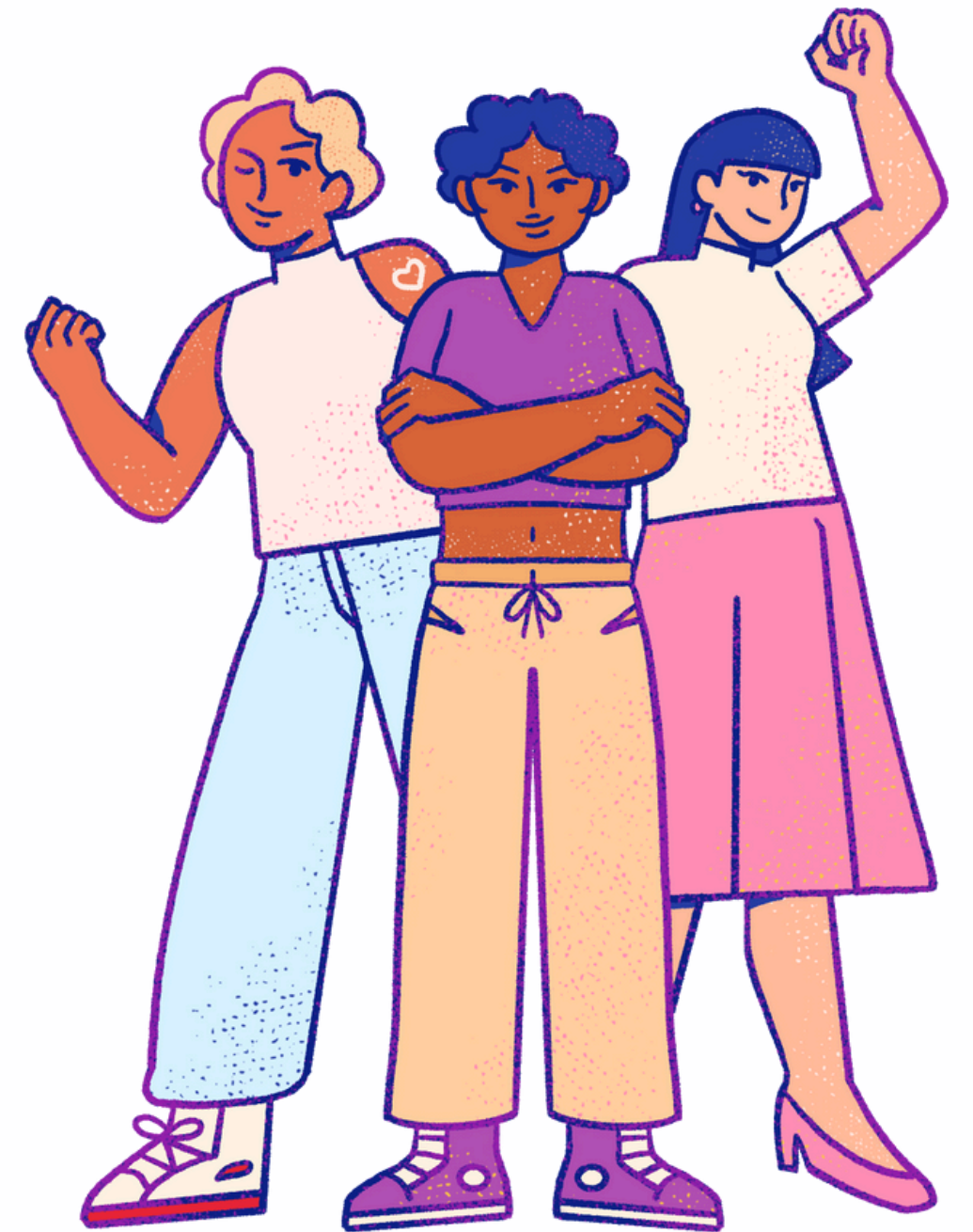
SHARED GO

MVP Release Presentation



Meei The Team!

Jingxiang Zhang
Soko Sengesamba
Minzhe Hu
Thibault Crosnier
Oliver Clease





AGENDA

- Project Overview
- Who is our Client
- User Stories
- Tech stack
- Design
- Demo
- Backend
- Problems & Solutions
- Client Feedback
- Future Plan
- Q&A



Real-time
activity map.

Project Overview



Progressive
Web App.



Easily create
and host
events.



Find events
near you.



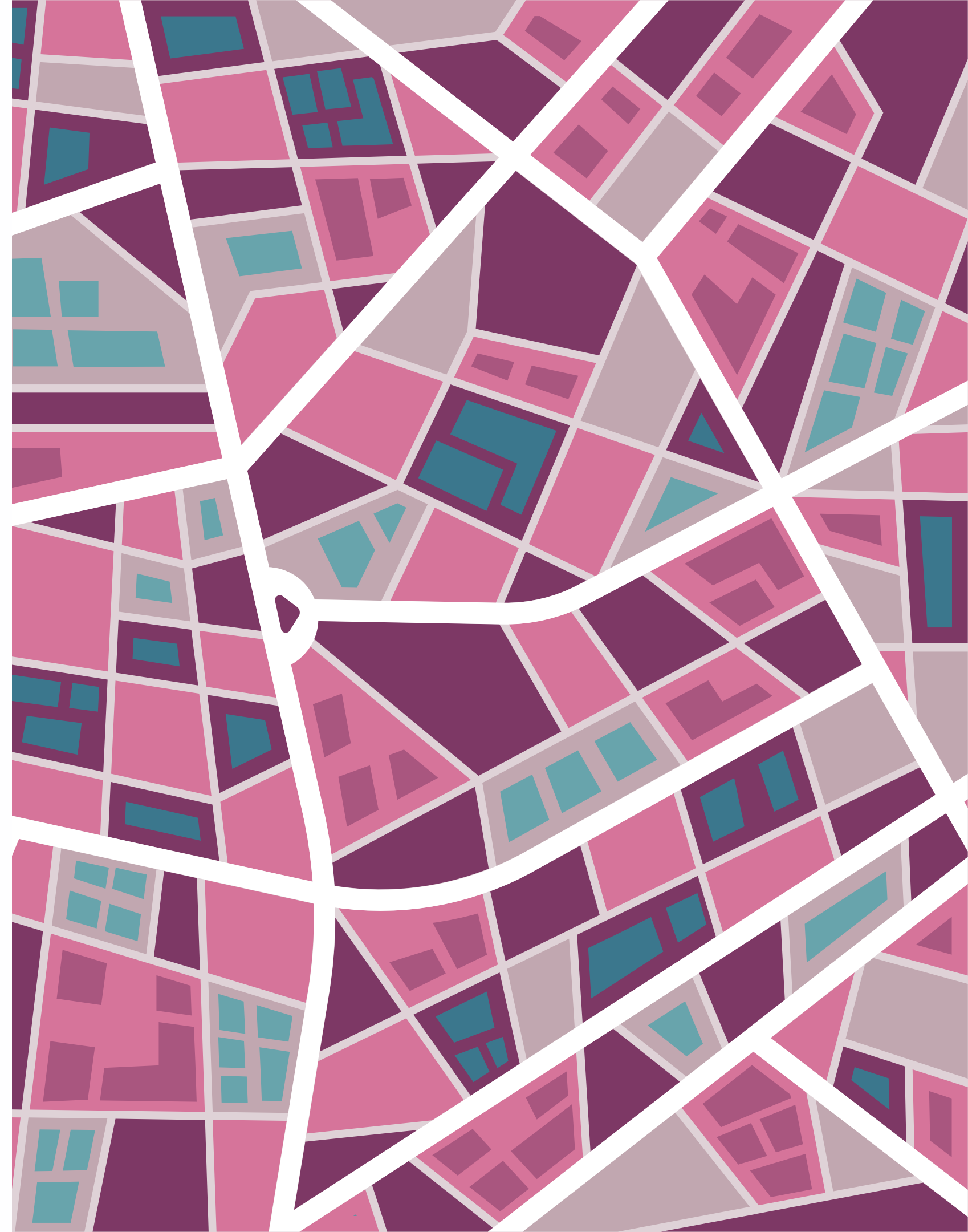
Join your
friends.

Our Client

MARIUS JURT

Marius' envisioned use-cases for Shared-Go include;

- Informal meet-ups
- BBQ on the downs
- University society events
- Friends and Families organising private events
- Large event organisers (concerts, fairs, socials.)





User Stories

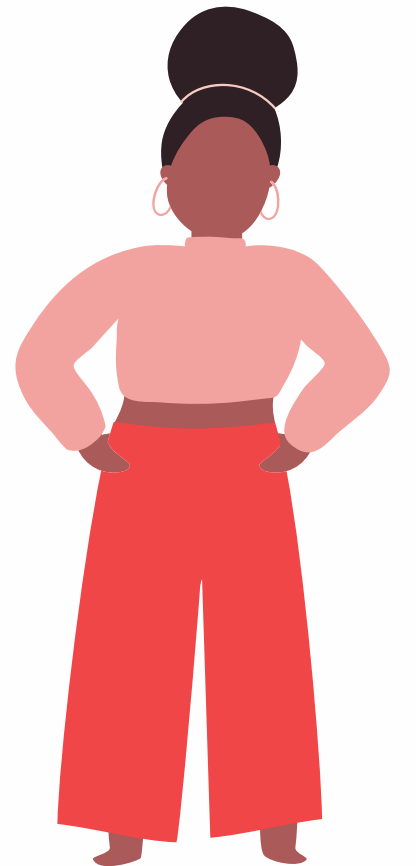
Event organiser

I want it to be easy to set up, manage and advertise events, that others can join.



Individual user

I want to have an integrated platform where I can quickly find events based on my location, friends and interests. I want to communicate effectively with organisers enriching my social life and event experience.

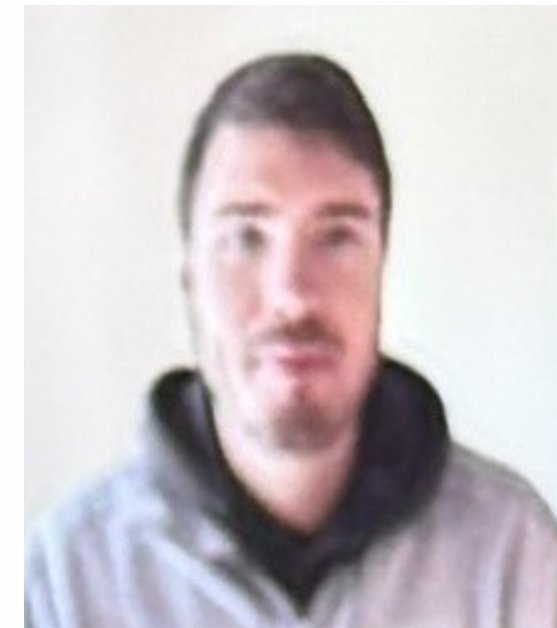


User Stories



Student societies

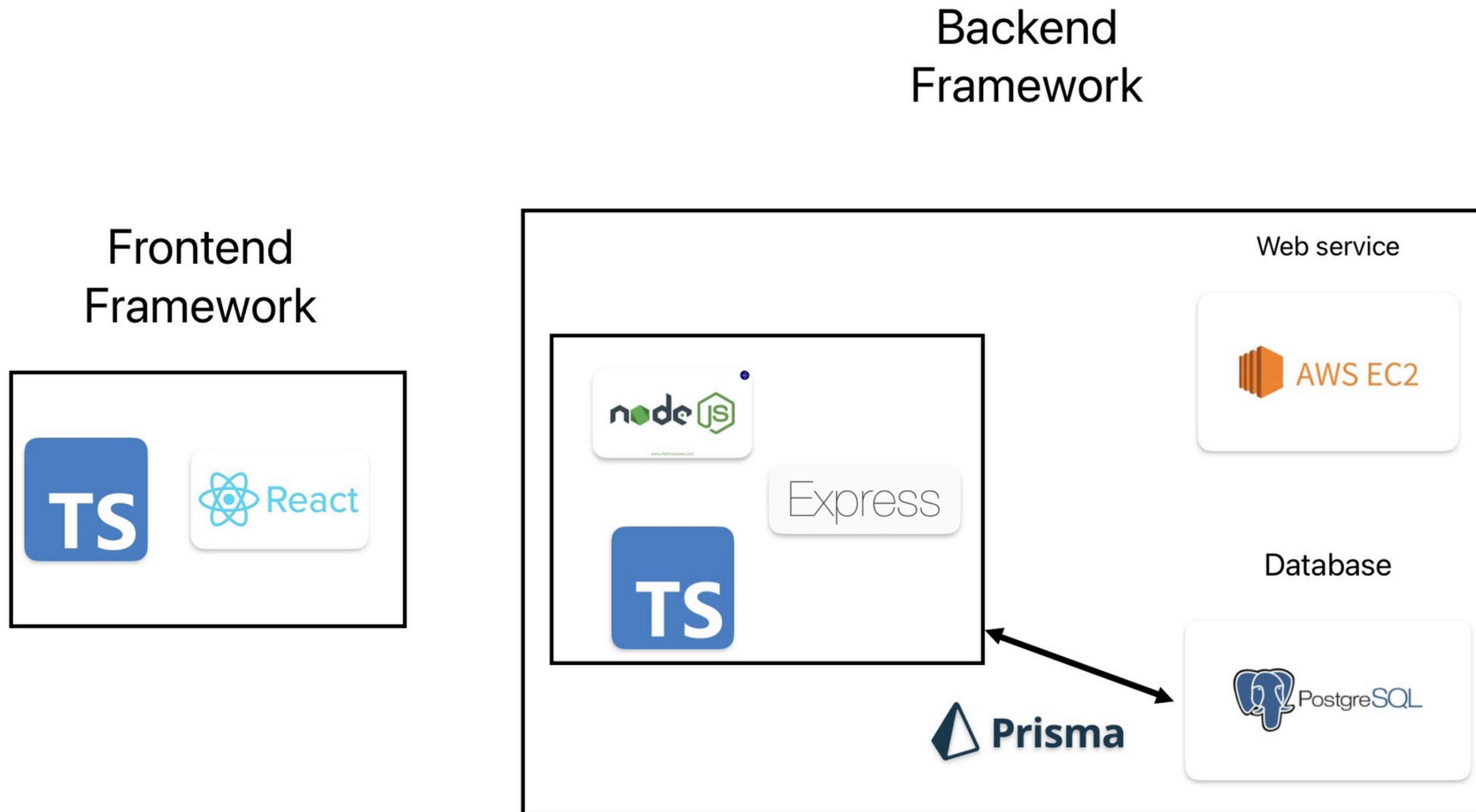
I want to be able to advertise all my events quickly and conveniently. I would like my society and its socials to be displayed to all users via the application and I want users to be able to join my society easily.



The Client

I want all users to have a way of setting-up informal meet-ups. The goal is "to build the most loved real-time activity map". I would like the app to positively impact groups, societies, event organisers etc..

Tech stack







DEMO

Of our frontend thus far

BACKEND

THUNDER CLIENT

New Request | v

Activity Collections Env

filter activity

GET localhost:3000/
just now

GET localhost:3000/api/users/3
2 months ago

POST localhost:3000/api/users
2 months ago

GET localhost:3000/api/users/3
2 months ago

GET localhost:3000/api/users/users/3
2 months ago

GET localhost:3000/api/users/users/3
2 months ago

GET localhost:3000/api/users
2 months ago

GET http://localhost:3000/events/1 Send

Query Headers² Auth Body¹ Tests

Query Parameters

☐	parameter	value
--------------------------	-----------	-------

Status: 200 OK Size: 773 Bytes Time: 686 ms

Response Headers⁶ Cookies Results Docs

```
1 {
2   "id": 1,
3   "title": "Community Meetup",
4   "description": "Hang out with people nearby and enjoy some snacks.",
5   "startsAt": "2025-10-10T17:00:00.000Z",
6   "capacity": 1234,
7   "seatsRemaining": 1233,
8   "location": "Central Park",
9   "imageUrl": "https://images.unsplash.com/photo-1466978913421-dad2ebd01d17?auto=
  =format&fit=crop&w=400&q=80",
10  "externalUrl": "https://example.com/sharego/community-meetup",
11  "host": {
12    "id": 1,
13    "name": "Host",
14    "email": "host@sharego.dev"
15  },
16  "attendees": [
17    {
18      "id": 2,
```

Copy

Response Chart

BACKEND

The screenshot shows a VS Code editor with several files open: .gitignore, TS seed.ts, TS events.ts, schema.prisma, package.json, TS hosts.ts, localhost:3000/, and TS index.ts. The REST client is open, showing a GET request to http://localhost:3000/hosts/1/reviews. The response is a JSON object with pagination and review data.

GET

Query Headers² Auth Body¹ Tests Pre Run

Query Parameters

parameter	value
-----------	-------

Status: 200 OK Size: 321 Bytes Time: 895 ms

Response Headers⁶ Cookies Results Docs

```
1 {
2   "pagination": {
3     "page": 1,
4     "limit": 10,
5     "total": 1
6   },
7   "reviews": [
8     {
9       "id": 1,
10      "rating": 5,
11      "comment": "Great atmosphere and friendly crowd.",
12      "createdAt": "2025-11-26T15:32:14.470Z",
13      "author": {
14        "id": 2,
15        "name": "First Attendee",
16        "email": "attendee@sharego.dev"
17      },
18      "event": {
19        "id": 1,
```

BACKEND

GET ⌵ http://localhost:3000/hosts/1/overview Send

Query Headers² Auth Body¹ Tests Pre Run

Query Parameters

☐ parameter value

Status: 200 OK Size: 655 Bytes Time: 626 ms

Response Headers⁶ Cookies Results Docs {} ≡

```
1 {
2   "host": {
3     "id": 1,
4     "name": "Host",
5     "email": "host@sharego.dev"
6   },
7   "stats": {
8     "totalEvents": 1,
9     "upcomingCount": 0,
10    "pastCount": 1,
11    "totalAttendees": 1,
12    "averageFillRate": 0,
13    "averageRating": 5,
14    "reviewCount": 1
15  },
16  "upcomingEvents": [],
17  "pastEvents": [
18    {
19      "id": 1,
```

Response Chart ↕

BACKEND

GET ⌵ http://localhost:3000/hosts/1/events Send

Query

Headers²

Auth

Body¹

Tests

Pre Run

Query Parameters

☐

parameter

value

Status: 200 OK Size: 58 Bytes Time: 229 ms

Response

Headers⁶

Cookies

Results

Docs

1

{

2

"pagination": {

3

"page": 1,

4

"limit": 10,

5

"total": 0

6

},

7

"events": []

8

}

Copy

Response

Chart

BACKEND TEST

PASS tests/events.test.ts

Event API

- ✓ should create a new event (129 ms)
- ✓ should return 400 when required fields are missing (19 ms)
- ✓ should join an event (15 ms)
- ✓ should not allow joining with missing email (15 ms)
- ✓ should not allow joining non-existent event (16 ms)
- ✓ should not allow joining a full event (11 ms)
- ✓ should return already joined message for duplicate participation (6 ms)
- ✓ should get event details (16 ms)
- ✓ should return 404 for non-existent event when getting details (14 ms)
- ✓ should return 400 for invalid event ID format (8 ms)

Test Suites: 1 passed, 1 total

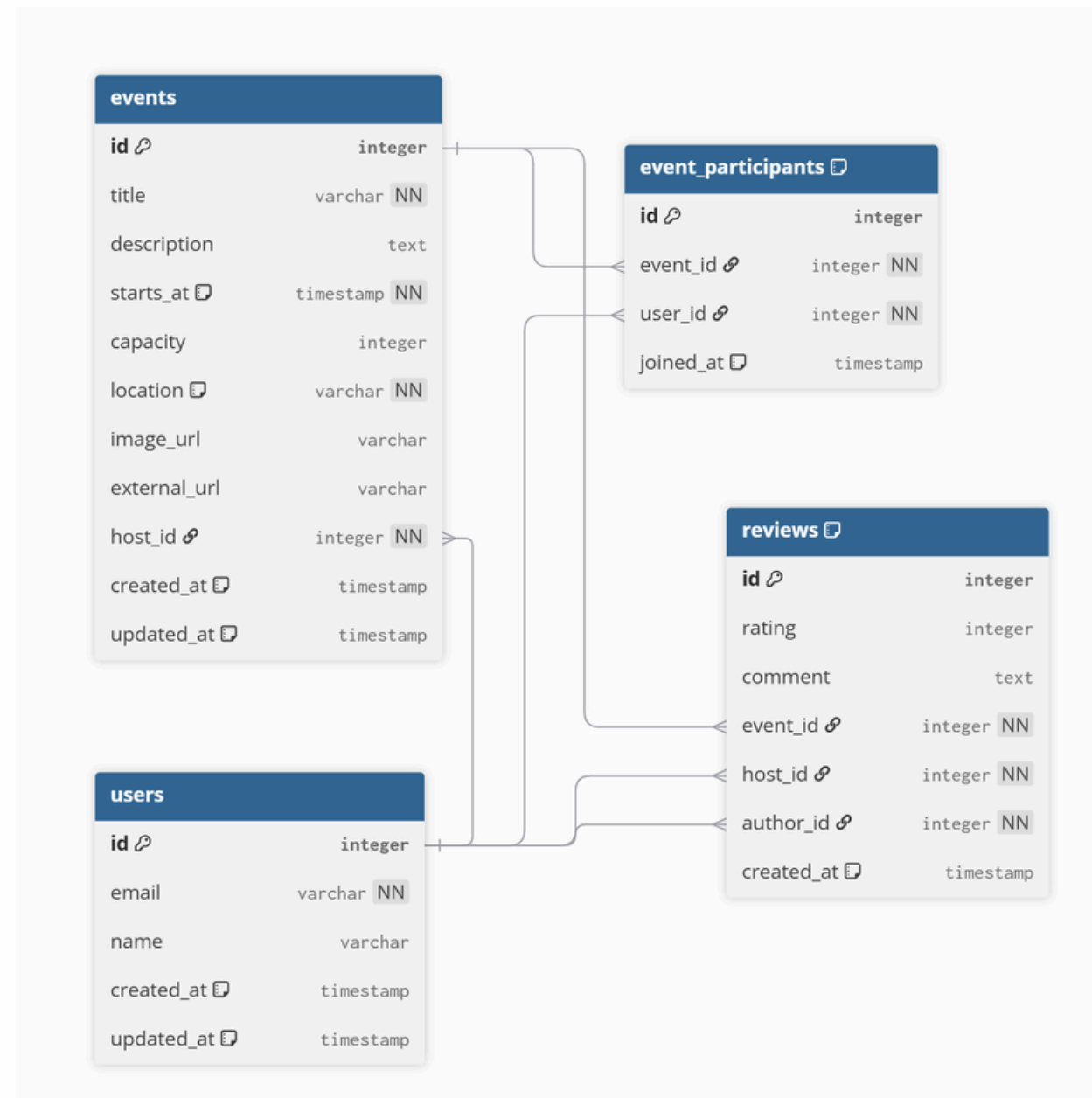
Tests: 10 passed, 10 total

Snapshots: 0 total

Time: 4.706 s

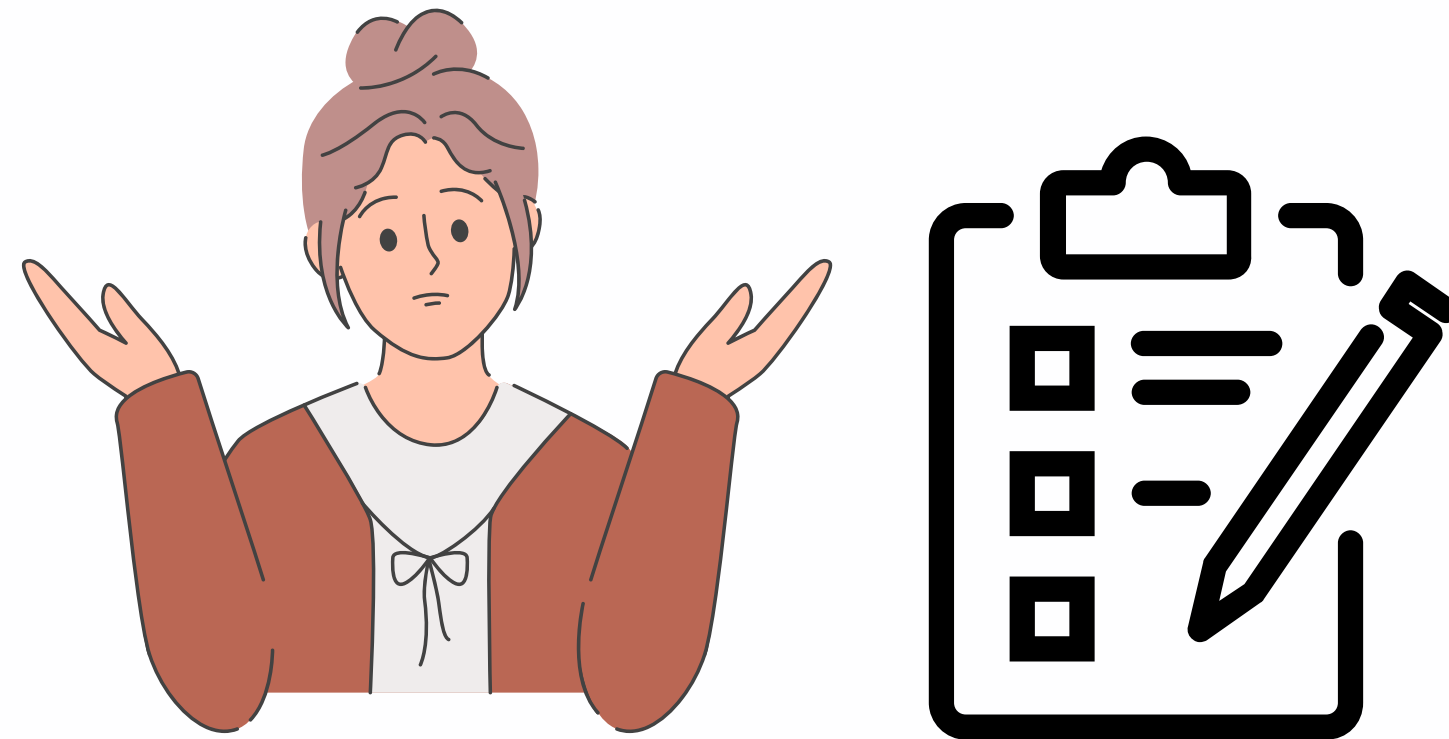
Ran all test suites.

DATABASE



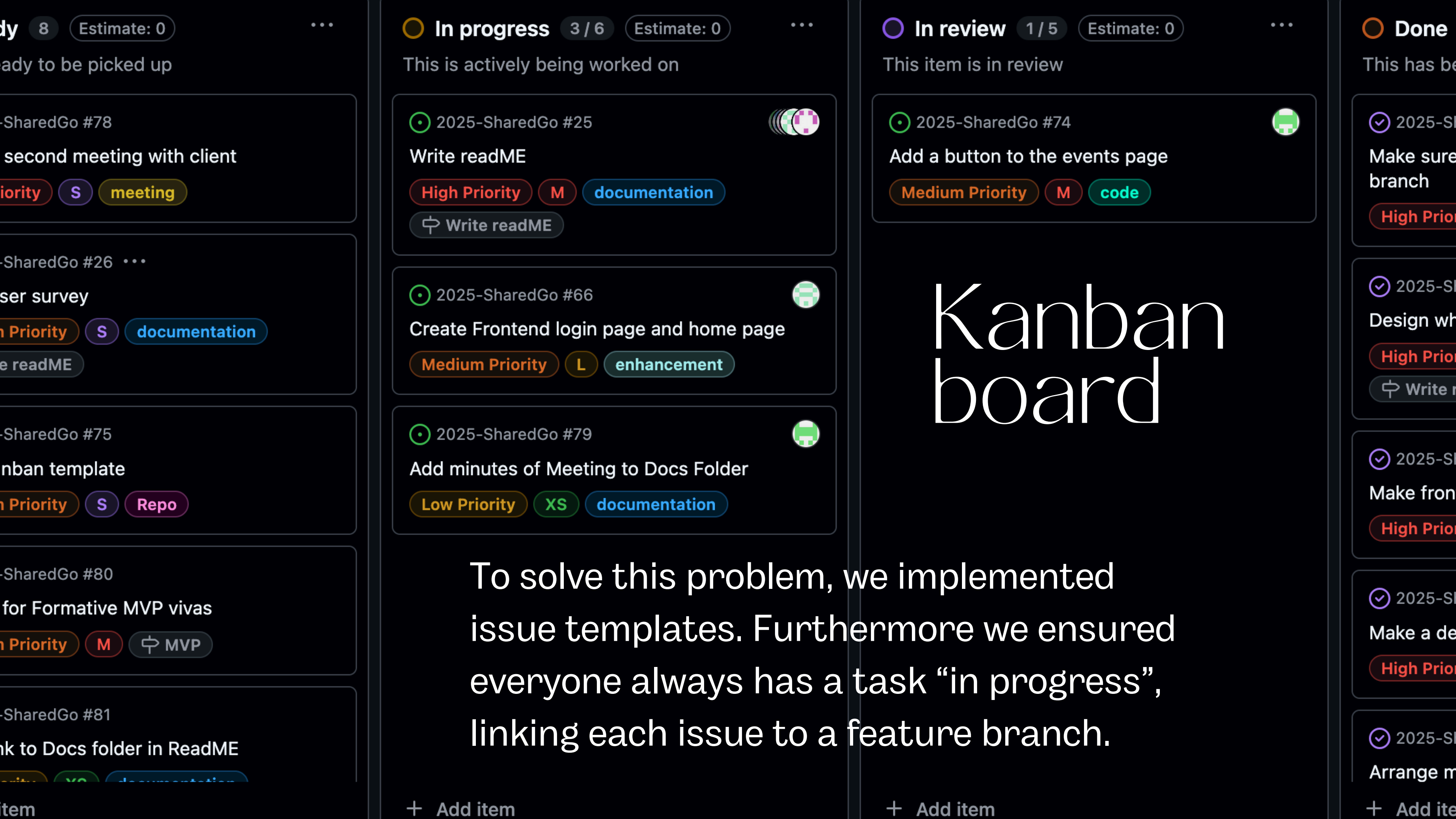
Search models		User X Event X Review X EventParticipant X Close all						
All Models								
Event	1							
EventParticipant	1							
Review	1							
User	2							
		Filters None Fields All Showing 2 of 2 Add record						
		☐	id #	email A	name A?	createdAt	updatedAt	eventsHosted []
		☐	1	host@sharego.dev	Host	2025-11-26T15:30:46.931Z	2025-11-26T15:30:46.931Z	1 Event
		☐	2	attendee@sharego.dev	First Attendee	2025-11-26T15:30:47.756Z	2025-11-26T15:30:47.756Z	0 Event

Problems + Solutions



VAGUE KANBAN BOARD

To begin with, our Kanban Issues were not very descriptive. We would struggle to understand what tasks each group member was working on.



Kanban board

To solve this problem, we implemented issue templates. Furthermore we ensured everyone always has a task “in progress”, linking each issue to a feature branch.

Problems + Solutions



COMMUNICATION

One problem that was encountered early on in the project was poor communication amongst our team.

The solutions we found were:

- > Regular meetings for discussion and catch-ups
- > Using Microsoft Teams to plan those meetings

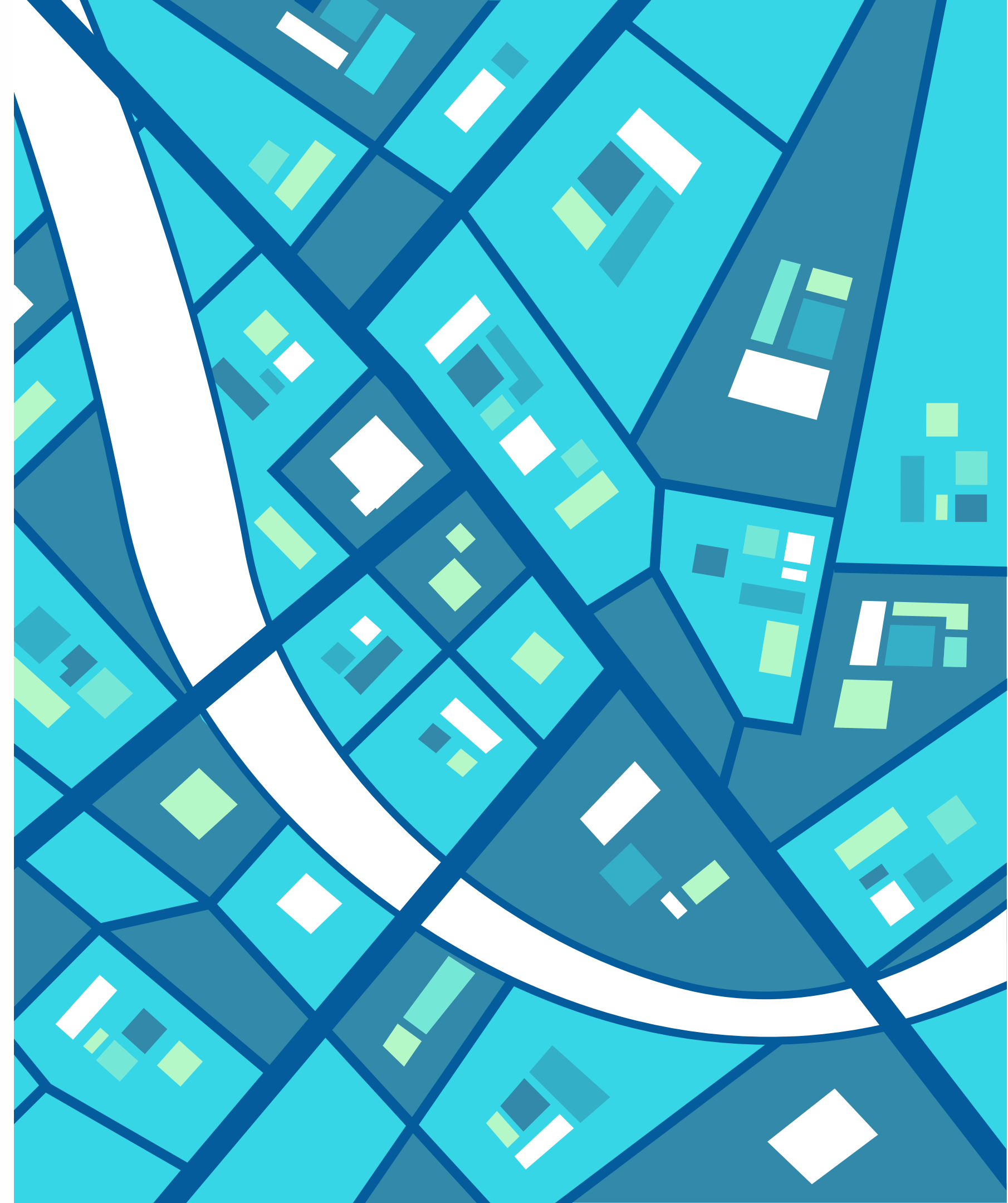
Client feedback

Positive feedback

Our client loves how simple the design is. It is nice and neat, which makes it intuitive for new users. In addition, UI can be inherited as the app has a unified design.

Timeline

Notifications, and reviews are useful, however, those can be implemented in the future. The same can be said for making the organiser messages more prominent in the chat page.



FUTURE PLAN

Connect front & back ends

Connect the frontend with the backend such that new events can correctly be created and joined.

Review system

Add a way for users to review events and hosts. This would enable a filter to be used on reputation.

1

Improve frontend looks

The frontend does not currently match our design.

2

3

Geolocation

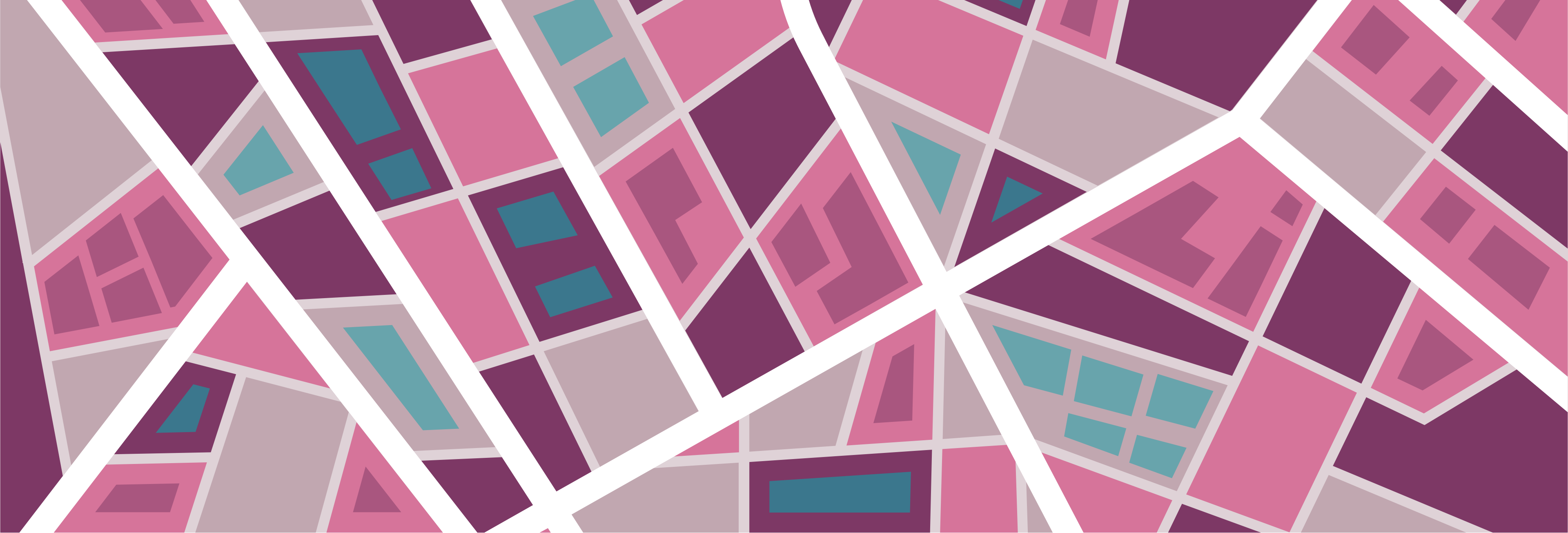
Such that users can search events based on their accessed location. This can be interfaced with the map.

4

5

Notifications

Implement notifications for users to be alerted when events are moved/cancelled or when their friends join an event.



THANK YOU! ✨



Q&A